A DEVELOPER'S DISCOVERY

My Windows Laptop Kept Crashing. Here's What I Learned.

A personal discovery about OS memory efficiency, GPU management, and why sometimes the best upgrade isn't more hardware.

Windows vs Linux | Memory Management | GPU Optimization | Developer Setup

Lim Chun Zhe

THE PROBLEM

OOM Crashes.
Every. Single. Day.

VS Code Devcontainers need Docker. On Windows, that means WSL2.
My laptop kept running out of memory and crashing.

MY SETUP

RAM: 40 GB • **GPU:** RTX 3060 Laptop • **OS:** Windows 11

Workload: VS Code Devcontainers + Browser + Video Editing

FIRST INSTINCT

"I need more RAM." → Ordered 32 GB upgrade.

But RAM isn't cheap — **especially these days.**

There has to be a more efficient way.

Instead of throwing hardware at the problem, I decided to understand **why** 40 GB wasn't enough.

THE DISCOVERY

Not All OS Memory Is Created Equal

Idle RAM usage — before opening any apps

1 GB

Ubuntu idle usage
Modular kernel, minimal services

5 GB+

Windows 11 idle usage
SysMain, telemetry, caching

~4 GB

macOS idle usage
Pre-caching & frameworks

2–5x

Linux is 2–5x more efficient
depending on config & total RAM

THE REAL COMPARISON

Windows vs WSL vs Native Linux

I needed Docker for VS Code Devcontainers. Here's the cost of each path.

Windows + Docker Desktop

Hyper-V VM layer
Docker Desktop overhead
Windows services in background

6–8 GB
with default settings

Windows + WSL2

Lightweight VM (default Docker backend)
Shared kernel, but still virtualized
vmmem can leak memory over time

~6 GB
Windows + WSL VM overhead

Native Ubuntu Linux

Docker runs natively on kernel
No VM, no translation layer
Direct hardware access

~1 GB
OS only, Docker adds minimal

FEATURE	WINDOWS	WSL2	NATIVE LINUX
Docker overhead	Hyper-V VM	Lightweight VM	Native kernel
File I/O speed	Slow (cross-FS)	Fast in /home	Native speed
GPU passthrough	Limited	CUDA yes, Vulkan partial	Full access
Memory reclaim	OS controls	Can leak (vmmem)	Full control

Devcontainers on WSL2 work, but Windows + WSL VM eats RAM before your container even starts. On native Linux, **Docker talks directly to the kernel** — no VM tax, no memory leaks.

GPU SURPRISE

Your iGPU Is Sitting Idle. Use It.

Most laptops have two GPUs. Most people only use one.
Linux lets you assign processes to each GPU individually.

DEFAULT (WASTEFUL)

VS Code, GNOME, browser —
all running on the **RTX 3060**.
iGPU completely **unused**.

20%

dGPU wasted on desktop UI

OPTIMIZED (BOTH GPUS WORKING)

Desktop + editors on **Intel iGPU**.
RTX 3060 reserved for
rendering, compute & local LLM inference.

0%

dGPU free until you need it

```
# Assign VS Code to Intel iGPU
$ env DRI_PRIME=0 code-insiders

# Set system to on-demand GPU switching
$ sudo prime-select on-demand
```

Fair note: Windows can also set per-app GPU via *Settings > Display > Graphics* or NVIDIA Control Panel. I haven't tried it myself — but it's a hint, not enforcement, and apps can override it. Linux's `DRI_PRIME` works at the driver level, making it harder to ignore.

THE REAL AHA MOMENT

Your Monitor Cable Chooses Your GPU

I discovered my laptop's ports are wired to *different* GPUs.

🔌

HDMI Port

Wired to **NVIDIA RTX 3060**

GPU stays active at all times

More heat, more power draw

→

⚡

USB-C Port

Wired to **Intel iGPU**

NVIDIA can fully power down

Less heat, GPU free for work

```
$ ls -lh /sys/class/drm/card*
card0 NVIDIA → HDMI-A-1, eDP-2
card1 Intel  → DP-1, DP-2, eDP-1
```

The problem: When I set apps to render on the iGPU but displayed through HDMI (NVIDIA-wired), frames had to copy across GPUs via *Reverse PRIME*. That caused **stuttering**.

The fix: Swapped to USB-C (Intel-wired). Now the iGPU renders *and* outputs directly — no frame copying. RTX 3060 dropped to **0%**.

Sometimes the best optimization is just **changing a cable**.

THE PROOF

RTX 3060 at 0%. Intel iGPU Handling It.

Actual screenshot from my Ubuntu machine after the changes.

0%	14%	15W	58°C
RTX 3060 utilization	Intel iGPU Render/3D	of 115W max power	Cool & quiet

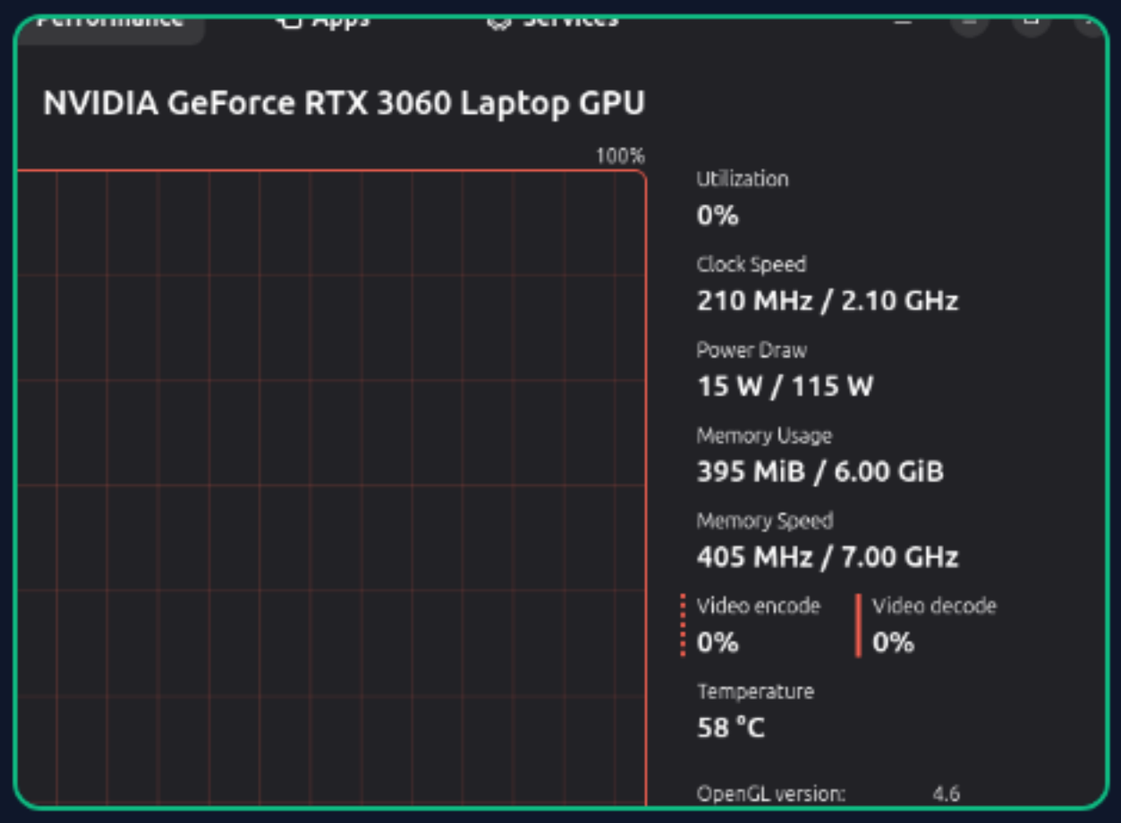
INTEL_GPU_TOP — INTEL IGPU RENDERING THE DESKTOP AT 14.41%

```
chunzhe@chunzhe-ONYX-V: ~/Deskto

intel-gpu-top: Intel Alderlake_p (Gen12) @ /dev/dri/card1 -  502/ 648 MHz;

         ENGINES     BUSY
      Render/3D    14.41% |
        Blitter     0.00% |
          Video     0.00% |
    VideoEnhance    0.00% |
```

MISSION CENTER — RTX 3060 AT 0%



NVIDIA-SMI — 56 MIB / 6144 MIB, P8 (SLEEPING)

```
| GPU  Name                 Persistence-M | Bus-Id
| Fan  Temp   Perf          Pwr:Usage/Cap |          Me
|                                         |
|=========================================+============
|   0  NVIDIA GeForce RTX 3060 ...    Off |   00000000:0
| N/A   58C    P8             15W /  115W |      56MiB /
|                                         |
+-----------------------------------------+------------
```

KEY TAKEAWAYS

What I Learned Today

💡 **Question the OS, not just the hardware.**
Linux uses ~1 GB idle vs Windows' 5 GB+. That's 4 GB back for your containers.

🎮 **Know where your GPU power goes.**
A text editor doesn't need an RTX 3060. Linux lets you choose per-app.

🔌 **Check your cable wiring.**
HDMI vs USB-C can mean the difference between GPU always-on and GPU idle.

🤔 **The best upgrade is understanding your system.**
Before buying more hardware, ask: is the software being efficient?

Have you tried dual-booting for development?

Let me know your setup in the comments 👇